

Tribhuvan University

Faculty of Humanities and Social Sciences

Masters of Computer Application (MCA)

1st Semester

Advanced Database Management System(MCA-503)

Unit-1: Object and Object Relational Databases 5hrs

Instructor

Tekendra Nath Yogi (M.SC CSIT TU)

Tekendranath@gmail.com

Contents

- **Unit 1: Object and Object-Relational Databases [5 Hrs.]**
 - 1.1. Overview of Object-Oriented concepts;
 - 1.2. Object database extension to SQL;
 - 1.3. The ODMG object model and the Object Definition Language (ODL);
 - 1.4. Object Database Conceptual Design;
 - 1.5. The Object Query Language (OQL)

Introduction to Database

- **Data:**
 - Known facts that can be recorded and have an implicit meaning.
- **Database:**
 - A collection of related data.
- **Database Management System (DBMS):**
 - A software system to facilitate the creation and maintenance of a computerized database.
- **Database System:**
 - The DBMS software together with the data itself. Sometimes, the applications are also included.

Cont'd...

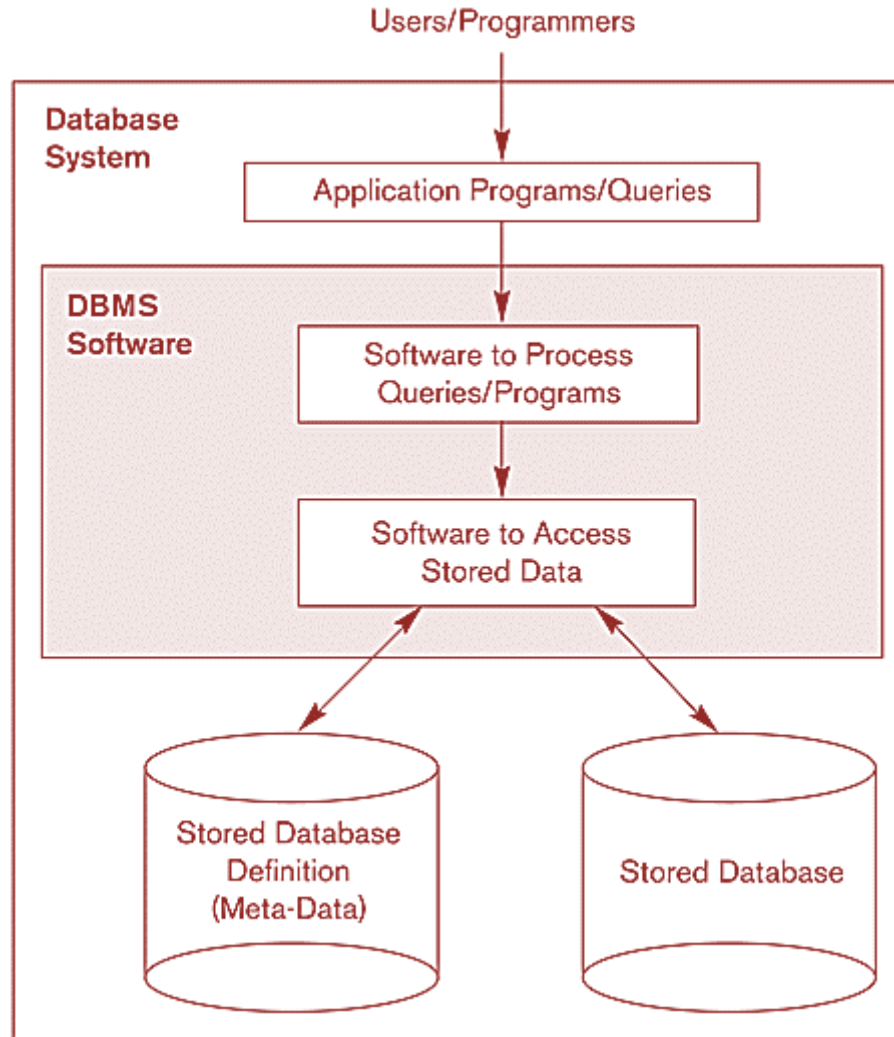
- **Database systems** are used to manage collections of data that are:
 - Highly valuable
 - Relatively large
 - Accessed by multiple users and applications, often at the same time.
- A **modern database system** manages a large, complex collection of data.
- Databases touch all aspects of our lives

Cont'd...

- **Types of Databases and Database Applications:**
 - **Traditional applications:** Numeric and textual databases like enterprise information databases (sales, accounting, HR database), manufacturing database, banking and finance database, University databases etc.
 - **Recent applications:** Multimedia databases, Geographic Information Systems (GIS), Biological and genome databases, Data warehouses, Mobile databases, Real-time and active databases.
 - **More Recent applications:** NOSQL and Big database for the social media data, search engine data, cloud data.

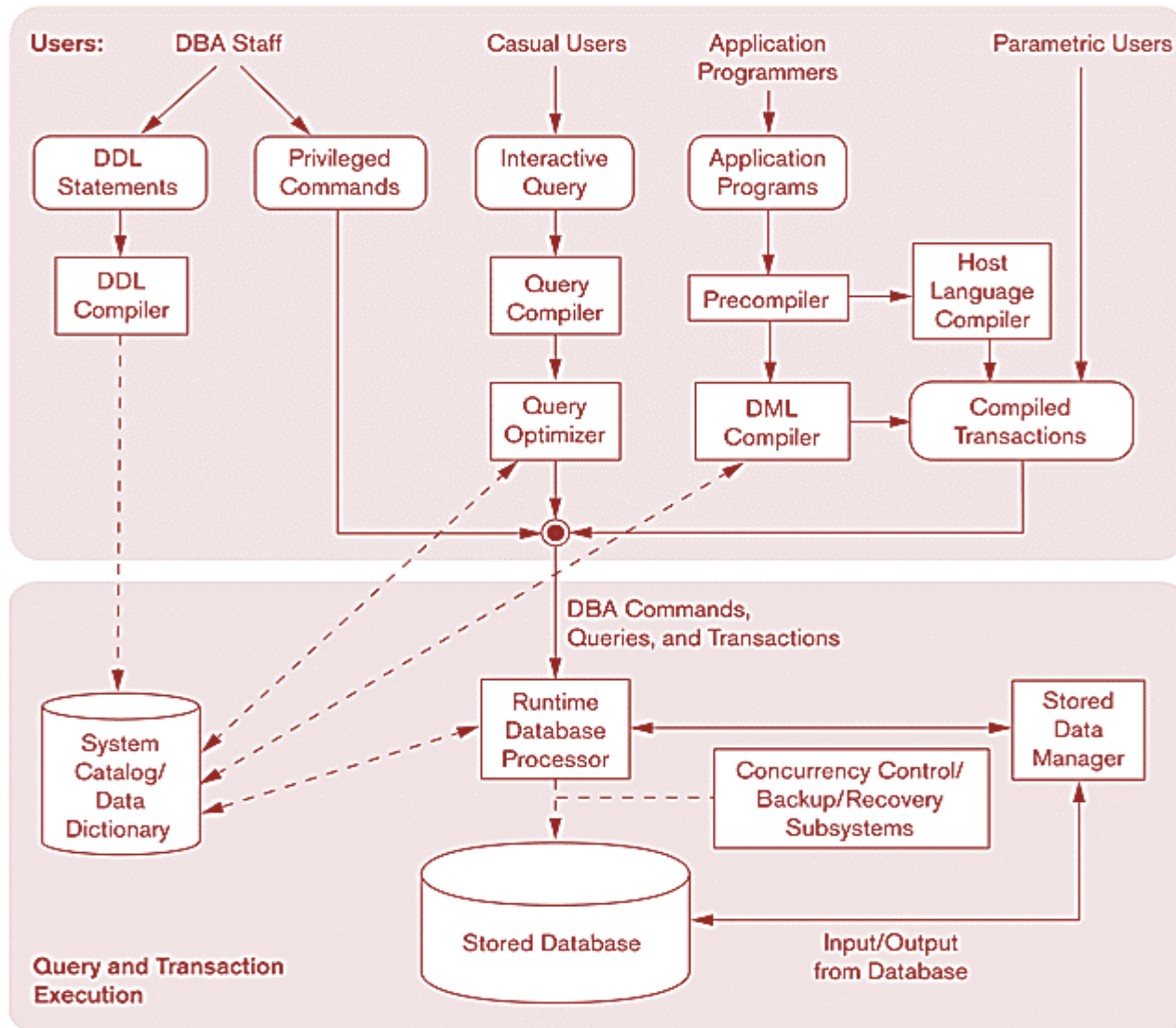
Cont'd...

- Simplified database system environment/Architecture:**



Cont'd...

- DBMS Component Modules:**



Cont'd...

- **DBMS Functionality:**
 - *Define a particular database* in terms of its data types, structures, and constraints
 - *Construct or Load the initial database contents on a secondary storage medium*
 - *Manipulating the database:*
 - **Retrieval:** Querying, generating reports
 - **Modification:** Insertions, deletions and updates to its content
 - **Accessing** the database through Web applications

Cont'd...

- *Processing and Sharing by a set of concurrent users and application programs – yet, keeping all data valid and consistent*
- *DBMS may additionally provide the following functionalities:*
 - Protection or Security measures to prevent unauthorized access
 - Active processing to take internal actions on data
 - Presentation and Visualization of data
 - Maintaining the database and associated programs over the lifetime of the database application

Cont'd...

- **Main Characteristics of the Database Approach:**
 - **Self-describing nature of a database system:**
 - Meta-data
 - **Insulation between programs and data:**
 - program-data independence.
 - **Data Abstraction:**
 - via conceptual, logical and physical models of the data.
 - **Support of multiple views of the data:**
 - Depending on the users interest.
 - **Sharing of data and multi-user transaction processing:**
 - via transaction processing, concurrency control and Recovery.

Cont'd...

- **Advantages of Using the Database Approach:**
 - Controlling redundancy in data storage and in development and maintenance efforts. That is, **Sharing of data among multiple users.**
 - Restricting unauthorized access to data.
 - Providing persistent storage for program Objects - **In Object-oriented DBMSs**
 - Providing Storage Structures (e.g. indexes) for efficient Query Processing
 - Providing backup and recovery services.
 - Providing multiple interfaces to different classes of users.
 - Representing complex relationships among data.
 - Enforcing integrity constraints on the database.
 - Drawing inferences and actions from the stored data using deductive and active rules

Cont'd...

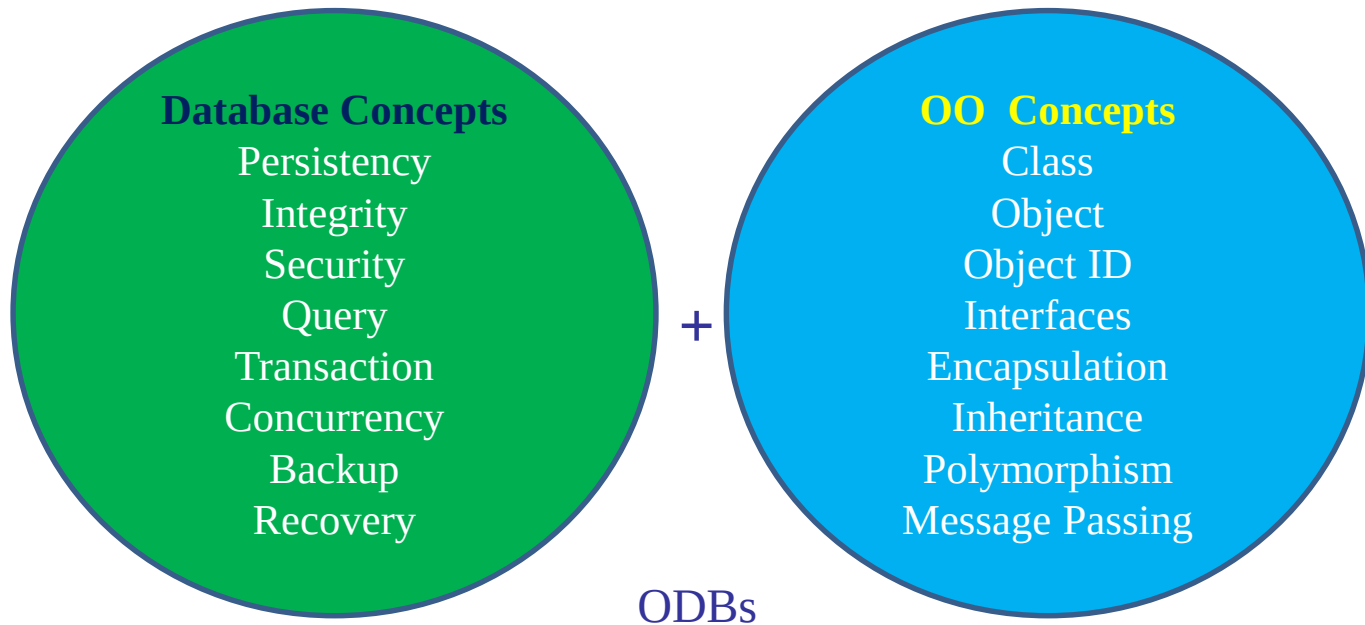
- **Data Models:**
 - Network Model
 - Hierarchical Model
 - Relational Model
 - Object-oriented Data Models
 - Object-Relational Models

Cont'd...

- **Classification of DBMSs: Based on the data model used**
 - **Legacy:** Network, Hierarchical.
 - **Currently Used:** Relational, Object-oriented, Object-relational
 - **Recent Technologies(NOSQL):** document based, column-based, graph-based and key-value based DBMSs.

1. Object and Object Relational Databases

- Database systems based on the **object data model** is known as **object-oriented databases (OODBs)** or **object databases (ODBs)**.
- Stores data in the form of **objects**, as in Object Oriented Programming Languages (OOPs).



- Software to Manage the ODBs is Known as Object Data Management System (ODMS) or ODBMS or OODBMS.

Cont'd...

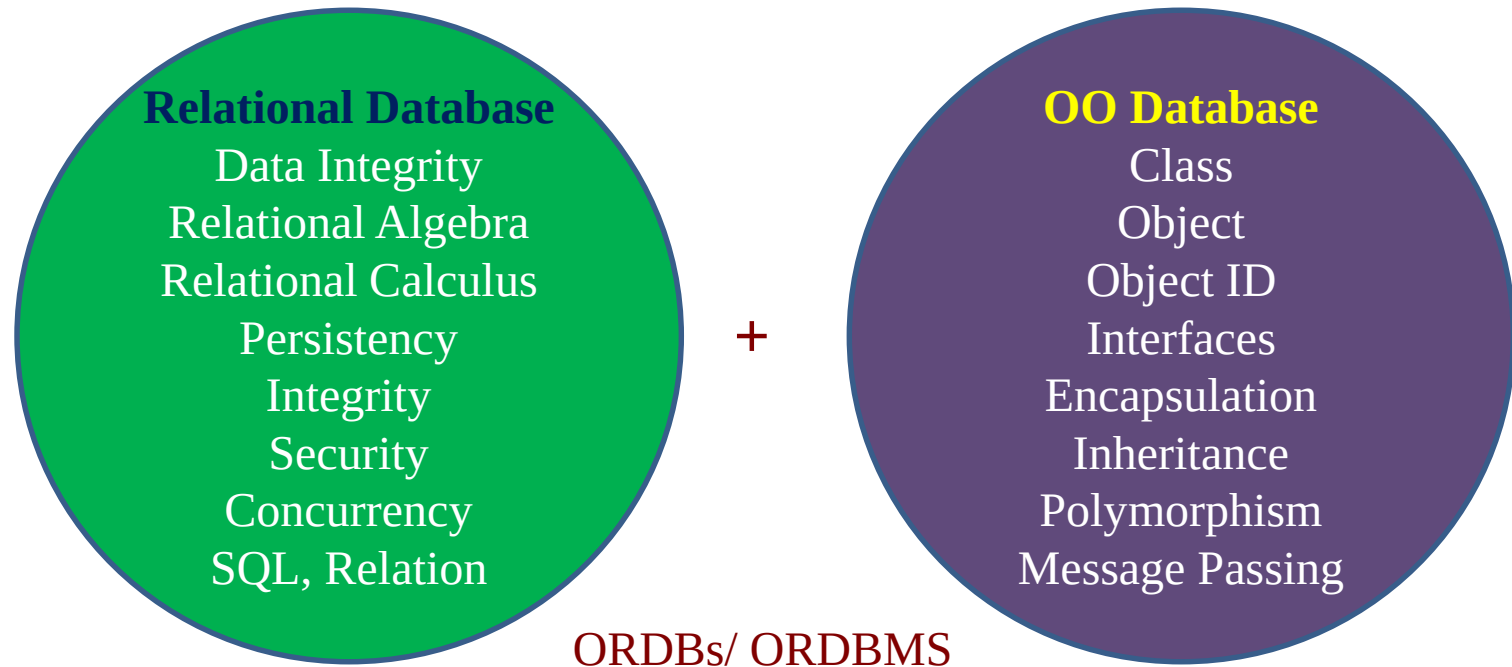
- Object Oriented (OO) Data Models and systems was developed since mid-90's.
- Reasons for creation of Object Oriented Databases
 - Need for more **complex applications** like CAD, Biomedical, GIS, Multimedia, etc.
 - Need for **additional data modeling features** like complex objects, inheritance, encapsulation, etc.
 - Increased use of **object-oriented programming languages**: ODBs for seamless integration of database with OOP based programs.
- An ODBs Standard was Developed by a consortium of object DBMS vendors and users, called Object Data Management Group (ODMG) - current specification is known as the ODMG 3.0 standard.

Cont'd...

- Several ODB products in the 1990's:
 - Experimental Systems: Orion, Iris, Open-OODB, etc.
 - Commercial OO Database products: Ontos, Gemstone, O2, Objectivity, Versant, Poet, etc.
- But did not make much impact on mainstream data management.
- Dominated by RDBMS due to the high popularity of RDBMS.
- However, **ODBs provides** Better modeling of real-world entities, Supports complex data types, Encapsulation of data and methods together. Reusability via inheritance, Improved performance for complex applications
- Hence, ODBs become the basis for various emerging databases like ORDBMS, NOSQL and Bigdata database.

Cont'd...

- An **Object Relational Databases (ORDBs)** combines **Relational database** features such as tables, SQL and **Object-oriented database** features such as object, inheritance etc.



- Examples of ORDBMS: Oracle Database and PostgreSQL

Cont'd...

- **Importance of ORDBMS:**

- Bridges gap between OOP and RDBMS.
- Supports complex applications such as GIS, multimedia, scientific systems
- Backward compatibility with RDBMS such as works with existing SQL systems
- Can handle both simple and complex data.
- Widely used in modern systems.

1.1. Overview of Object Database Concepts

- **Object databases (ODBs)** Origins in OOPLs (Object Oriented programming languages) such as SIMULA, Smalltalk, etc.
- Includes the following object-oriented features:
 - **Object:** Instance Variable (State – value) , Operations (behavior) – signature and Implementation
 - **Encapsulation:** bundles data and Operations together.
 - **Message Passing:** accessing object state through operations by passing arguments.
 - Type and class hierarchy and **Inheritance**
 - **Relationship** in the form of reference/ Inverse reference using OID.
 - **Polymorphism:** Operator/Operation overloading

Cont'd...

- **1.1.1. Object:**

- An object is a real-world entity with two components State (value) and behavior (operations).
 - **Instance variables** (attributes): Hold values that define internal state of object
 - **Operation** is defined in two parts: Signature (interface) and implementation (method or body) to enforce encapsulation
- Two types of objects:
 - **Transient Object:** Exists only during program execution. For example, OOPL Program objects.
 - **Persistent Object:** Stored in database and exists after program ends. For example, ODBs objects stored in the disk
- An ODBs stores objects permanently and allows sharing across applications.

Cont'd...

- **1.1.2. Object Identity, and Objects versus Literals: Object Identity**
 - To make the direct correspondence of real world objects in ODBs has Unique identity called Object Identity (OID).
 - Implemented via a **unique, system-generated** object identifier (OID).
 - **Immutable** - should not change even after object removed from the ODBs.
 - OID should not depend on any attribute values of the object.
 - OID should not be based on the physical address of the object in storage.

Cont'd...

- **1.1.2. Object Identity, and Objects versus Literals: OID in ODBs vs Primary Key in RDBs:**

OID	Primary Key (PK)
1. Automatically generated by the system.	1. Defined by the user or database designer.
2. Hidden from users and used internally.	2. visible and used in queries.
3. Never changes once it is assigned.	3. value can be changed.
4. Does not depend on object data values.	4. based on attribute values.
5. Uniquely identifies objects permanently.	5. may change and affect identity.

Cont'd...

- **1.1.2. Object Identity, and Objects versus Literals: Objects versus Literals:**
 - Most OO database systems allow for the representation of both **objects** and **literals** (simple or complex values).
 - Every object must have an immutable OID, whereas a literal value has no OID and its value just stands for itself.
 - Thus, a literal value is typically stored within an object and cannot be referenced from other objects.

Contd.,

- **1.1.2. Object Identity, and Objects versus Literals: Objects versus Literals:**

Objects	Literals
1. Has a unique identifier (OID).	1. Does not have any identifier.
2. Exists independently in the database.	2. Exists only as a value inside an object.
3. Can be referenced by other objects.	3. Cannot be referenced by other objects.
4. Represents complex entities.	4. Represents simple values like numbers or strings.
5. Has both state and behavior.	5. Only represents a value.

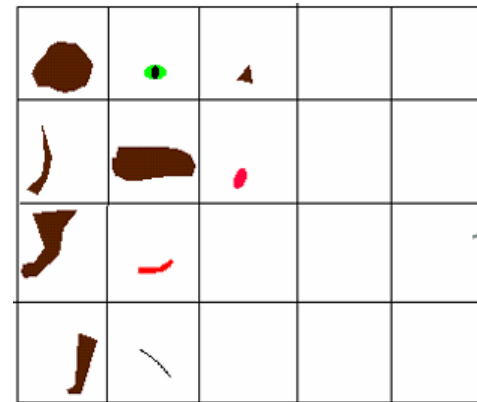
Cont'd...

- **1.1.3. Complex Type Structures for Objects and Literals:**

- In ODBs, objects and literals may have a **type structure of arbitrary complexity** in order to contain all of the necessary information that describes the object or literal.



ODB of a cat



RDB of a cat

- In contrast, in traditional database systems, information about a complex object is often scattered over many relations or records, leading to loss of direct correspondence between a real-world object and its database representation.

Cont'd...

- In ODBs, a **complex type** may be constructed from other types by **nesting** of type constructors.
- The three most **basic type constructors** (**type generators**) are:
 - **Atom (Single Valued)**
 - **Struct (or tuple)**
 - **Collection (or multi-valued)**

Cont'd...

- **Atom (single valued, Atomic types) Constructor:**
 - Atom constructor includes the basic built in data types of the object model, which are similar to the basic types in OOPL.
 - E.g. integers, strings, floating-point numbers, enumerated types, Booleans, and so on.

Cont'd...

- **Struct (or tuple) type constructor:**
 - Can create standard structured types, such as the **tuples** (record types) in the basic relational model.
 - made up of several components and is also sometimes referred to as a **compound** or **composite** type.
 - Is a type generator rather than a type.
 - because many different structured types can be created.
 - For example, two different structured types that can be created are:
 - `struct Name<FirstName: string, MiddleInitial: char, LastName: string>`,
and
 - `struct CollegeDegree<Major: string, Degree: string, Year: date>`.

Cont'd...

- **Collection (or Multivalued) type Constructors:** Used to Create complex nested type structures in the object.
- Includes the following type constructors
 - **Set(T):** Unique elements, e.g., set(EMPLOYEE).
 - **Bag (T):** Elements may not be unique, e.g., bag(integer).
 - **List (T):** Ordered set of elements, e.g., list(STUDENT).
 - **Array (T):** Fixed collection of indexed elements e.g. Array(integer).
 - **Dictionary(K,T):** Elements are in key value pair E.g., dictionary(string : string)
- **Object definition language (ODL)** that incorporates the above type constructor is used to define object types for a particular database application.

Cont'd...

- Figure 1.1 Specifying the object types EMPLOYEE, DATE, and DEPARTMENT using type constructors.);

define type EMPLOYEE

```
tuple (  Fname:      string;
         Minit :     char;
         Lname:      string;
         Ssn:        string;
         Birth_date: DATE;
         Address:    string;
         Sex:        char;
         Salary:     float;
         Supervisor: EMPLOYEE;
         Dept:       DEPARTMENT; );
```

define type DATE

```
tuple (  Year:      integer;
         Month:     integer;
         Day:       integer; );
```

define type DEPARTMENT

```
tuple (  Dname:      string;
         Dnumber:    integer;
         Mgr:        tuple (  Manager:      EMPLOYEE;
                             Start_date:    DATE; );
         Locations:  set(string);
         Employees:  set(EMPLOYEE);
         Projects:   set(PROJECT); );
```

Contd..,

- **1.1.4. Encapsulation of Operations: Encapsulation**
 - Related to abstract data types and information hiding.
 - Define **behavior** of a class of object based on operations that can be externally applied
 - External users only aware of interface of the operations
 - Can divide structure of object into **visible** and **hidden** attributes

Contd..,

- Typical operations that can be encapsulated inside objects includes the following:
 - **Constructor** operation
 - Used to create a new object
 - **Destructor** operation
 - Used to destroy (delete) an object
 - **Modifier** operations
 - Modify the state of an object
 - **Retrieve** operation
 - Read the state of an object
- Note: Dot notation to apply operations to object
 - E.g. O1.no_of_emps.

Contd.,

- Figure 1.2 Adding operations to the definitions of EMPLOYEE and DEPARTMENT.

```
define class EMPLOYEE
  type tuple (  Fname:      string;
                Minit:     char;
                Lname:     string;
                Ssn:       string;
                Birth_date: DATE;
                Address:   string;
                Sex:       char;
                Salary:    float;
                Supervisor: EMPLOYEE;
                Dept:      DEPARTMENT; );
  operations
    age:      integer;
    create_emp: EMPLOYEE;
    destroy_emp: boolean;
end EMPLOYEE;
```

```
define class DEPARTMENT
  type tuple (  Dname:      string;
                Dnumber:   integer;
                Mgr:       tuple ( Manager: EMPLOYEE;
                                   Start_date: DATE; );
                Locations: set (string);
                Employees: set (EMPLOYEE);
                Projects   set (PROJECT); );
  operations
    no_of_emps: integer;
    create_dept: DEPARTMENT;
    destroy_dept: boolean;
    assign_emp(e: EMPLOYEE): boolean;
    (* adds an employee to the department *)
    remove_emp(e: EMPLOYEE): boolean;
    (* removes an employee from the department *)
end DEPARTMENT;
```

Contd..,

- **Specifying Object Persistence via Naming and Reachability:**
 - **Transient objects**
 - Exist in executing program and disappear once program terminates
 - **Persistent objects**
 - Stored in database, persist after program termination
 - Two mechanisms for making an object persistent are:
 - **Naming mechanism:** object assigned a unique name in object database, user finds object by its name. But Database may contains thousands of objects so, not suitable.
 - **Reachability:** object referenced from other persistent objects, object located through references

Contd..,

- **Extent**
 - A *named persistent object* to hold collection of all persistent objects for a class.
 - Used as entry points to the database.
 - For example, ALL_DEPARTMENTS
- **Persistent collection**
 - Stored permanently in the database
- **Transient collection**
 - Exists temporarily during the execution of a program (e.g. query result)

Contd.,

- Figure 1.3 Creating persistent objects by naming and reachability.

```
define class DEPARTMENT_SET
  type set (DEPARTMENT);
  operations add_dept(d: DEPARTMENT): boolean;
    (* adds a department to the DEPARTMENT_SET object *)
    remove_dept(d: DEPARTMENT): boolean;
    (* removes a department from the DEPARTMENT_SET object *)
    create_dept_set:    DEPARTMENT_SET;
    destroy_dept_set:   boolean;
end Department_Set;

...
persistent name ALL_DEPARTMENTS: DEPARTMENT_SET;
(* ALL_DEPARTMENTS is a persistent named object of type DEPARTMENT_SET *)
...
d:= create_dept;
(* create a new DEPARTMENT object in the variable d *)
...
b:= ALL_DEPARTMENTS.add_dept(d);
(* make d persistent by adding it to the persistent set ALL_DEPARTMENTS *)
```

Contd..,

- **1.1.5. Type (Class) Hierarchies and Inheritance:**

- **Inheritance**

- Refers to the definition of new types based on other predefined types
 - Leads to **type** (or **class**) **hierarchy**

- **Type:**

- Is created by specifying **type_name** and list of visible (public) **functions** (attributes or operations)
 - Format:
 - TYPE_NAME: function, function, ..., function
 - Example:
 - PERSON: Name, Address,, Age, Salary.

Contd..,

- **Subtype**
 - Useful when creating a new type that is similar but not identical to an already defined type
 - Subtype inherits functions from the supertype.
 - Additional (local or specific) functions in subtype
 - Example:
 - EMPLOYEE subtype-of PERSON: Salary, Hire_date, Seniority
 - STUDENT subtype-of PERSON: Major, GPA

Contd..,

- **1.1.6. Other Object-Oriented Concepts: Polymorphism of operations**
 - Also known as **operator overloading**
 - Allows same operator name or symbol to be bound to two or more different implementations
 - Type of objects determines which operator is applied
 - E.g., + operator gives sum if operands are of type int, gives concatenated string if operands are string.
 - Achieved by using the operator overloading mechanism.
 - Binding of Operations occurs:
 - Early or static binding
 - Late or dynamic binding

Contd..,

- **1.1.6. Other Object-Oriented Concepts: Multiple inheritance**
 - Subtype inherits functions (attributes and operations) of more than one supertype
 - E.g, **Engineer_Manager** inherits features from **Manager** and **Engineer** supertypes.
 - **Problem:** Ambiguity of function. E.g. salary
 - Solution to deal with ambiguity:
 - During inheritance allow the user to choose which one to inherit.
 - Use system defaults
 - Disallow multiple inheritance all together.
- **Note:** Selective inheritance using **except** clause is not used in ODBs.

1.2. Object-Relational Features: Object DB Extensions to SQL

- SQL stands for Structure Query Language is a standard language for RDBMSs.
- SQL was first specified by Chamberlin and Boyce (1974) and underwent enhancements and standardization in 1989 and 1992.
- The language continued its evolution with a new standard, initially called SQL3 later known as SQL:99.
- Starting with the version of SQL known as SQL3, features from object databases were incorporated into the SQL standard.
- At first, these extensions were known as SQL/Object, but later they were incorporated in the main part of SQL, known as SQL/Foundation in SQL:2008.

Contd..,

- The following are some of the **object database features** that have been included in SQL:
 - **Type constructors (generators) to specify complex objects**
 - Row type / tuple (or struct) constructor.
 - Early SQL had only array for collections
 - Later versions of SQL added other collection types (ARRAY, MULTISSET, LIST, SET, etc.)
 - Mechanism for specifying **object identity**
 - Through the use of reference type
 - **Encapsulation of operations** provided through
 - User-defined types (UDTs) by allowing specifying attribute and operations together.
 - **Inheritance mechanisms**
 - Provided using keyword UNDER

Contd..,

- **1.2.1. User-Defined Types (UDTs) Using CREATE TYPE and Complex Objects:**

- The UDTs are created as part of the database schema using the following syntax:

```
CREATE TYPE <type name> AS  
(  
    <component declarations>  
);
```

- Here, components can be attributes and operations.

Contd.,

- UDTs Can be used to create a:
 - **Type for an attribute** (similar to struct – no operations) Or
 - **Type for a table of objects** (similar to class – can have operations)

Figure 1.4a Illustrating some of the object features of SQL. Using UDTs as **types for attributes** such as Address and Phone.

```
(a) CREATE TYPE STREET_ADDR_TYPE AS (  
    NUMBER          VARCHAR (5),  
    STREET          NAME VARCHAR (25),  
    APT_NO          VARCHAR (5),  
    SUITE_NO        VARCHAR (5)  
);  
CREATE TYPE USA_ADDR_TYPE AS (  
    STREET_ADDR     STREET_ADDR_TYPE,  
    CITY            VARCHAR (25),  
    ZIP             VARCHAR (10)  
);  
CREATE TYPE USA_PHONE_TYPE AS (  
    PHONE_TYPE      VARCHAR (5),  
    AREA_CODE       CHAR (3),  
    PHONE_NUM       CHAR (7)  
);
```

Contd..,

- **1.2.2. Object Identifiers Using Reference Types :**
 - To uniquely identify the object/tuple/rows in the table requires unique object identifier (OID).
 - SQL provides the **REF** keyword to create OIDs using reference type.
 - Two Approach:
 1. Can specify system-generated object identifiers.
 - For Example: **REF IS SYSTEM GENERATED**
 2. Alternatively can use primary key as OID as in traditional relational model
 - For Example: **REF IS <OID_ATTRIBUTE> <VALUE_GENERATION_METHOD> ;**
 - **VALUE_GENERATION_METHOD** can be **SYSTEM GENERATED** or **DERIVED**.

Contd..,

- **1.2.3. Creating Tables Based on the UDTs: `INSTANTIABLE`**
 - Specify that UDT is **instantiable**
 - The user can then create one or more tables based on the UDT
 - If keyword **INSTANTIABLE** is left out, can use UDT only as attribute data type – not as a basis for a table of objects

Contd..,

- **1.2.4. Encapsulation of Operations:** By using User-defined type
 - Specify methods (or operations) in addition to the attributes while creating user defined type.
 - **Format:**

CREATE TYPE <TYPE-NAME> AS

(

<LIST OF COMPONENT ATTRIBUTES AND THEIR TYPES>

<DECLARATION OF FUNCTIONS (METHODS)>

);

Contd..,

- Figure 1.4b Illustrating some of the object features of SQL. Specifying UDT for PERSON_TYPE.

```
(b) CREATE TYPE PERSON_TYPE AS (  
    NAME          VARCHAR (35),  
    SEX           CHAR,  
    BIRTH_DATE    DATE,  
    PHONES        USA_PHONE_TYPE ARRAY [4],  
    ADDR          USA_ADDR_TYPE  
  
    INSTANTIABLE  
    NOT FINAL  
    REF IS SYSTEM GENERATED  
    INSTANCE METHOD AGE() RETURNS INTEGER;  
    CREATE INSTANCE METHOD AGE() RETURNS INTEGER  
        FOR PERSON_TYPE  
    BEGIN  
        RETURN /* CODE TO CALCULATE A PERSON'S AGE FROM  
                TODAY'S DATE AND SELF.BIRTH_DATE */  
    END;  
);
```


Contd..,

- **1.2.5. Specifying Inheritance : Specifying Type Inheritance**
 - In SQL, inheritance can be applied to **types** or to **tables**
 - To create the **type inheritance** in SQL use the following two keywords:
 - **NOT FINAL:**
 - The keyword NOT FINAL indicates that subtypes can be created for that type
 - **UNDER:**
 - The keyword UNDER is used to create a subtype

Contd..,

- Figure 1.4c Illustrating some of the object features of SQL. Specifying UDTs for STUDENT_TYPE and EMPLOYEE_TYPE as two subtypes of PERSON_TYPE.

```
(c) CREATE TYPE GRADE_TYPE AS (  
    COURSENO    CHAR (8),  
    SEMESTER    VARCHAR (8),  
    YEAR        CHAR (4),  
    GRADE        CHAR  
);  
CREATE TYPE STUDENT_TYPE UNDER PERSON_TYPE AS (  
    MAJOR_CODE   CHAR (4),  
    STUDENT_ID   CHAR (12),  
    DEGREE       VARCHAR (5),  
    TRANSCRIPT   GRADE_TYPE ARRAY [100]
```

```
INSTANTIABLE  
NOT FINAL  
INSTANCE METHOD GPA() RETURNS FLOAT;  
CREATE INSTANCE METHOD GPA( ) RETURNS FLOAT  
    FOR STUDENT_TYPE  
    BEGIN  
        RETURN /* CODE TO CALCULATE A STUDENT'S GPA FROM  
                SELF.TRANSCRIPT */  
    END;  
);  
CREATE TYPE EMPLOYEE_TYPE UNDER PERSON_TYPE AS (  
    JOB_CODE     CHAR (4),  
    SALARY        FLOAT,  
    SSN           CHAR (11)  
INSTANTIABLE  
NOT FINAL  
);  
CREATE TYPE MANAGER_TYPE UNDER EMPLOYEE_TYPE AS (  
    DEPT_MANAGED CHAR (20)  
INSTANTIABLE  
);
```

Contd..,

- **1.2.5. Specifying Inheritance : Type inheritance rules:**
 - All attributes/operations are inherited
 - Order of supertypes in UNDER clause determines inheritance hierarchy
 - Instance (object) of a subtype can be used in every context in which a supertype instance used
 - Subtype can redefine any function defined in supertype

Contd..,

- **1.2.5. Specifying Inheritance: Table Inheritance:**
 - **UNDER** keyword can also be used to specify super-table / sub-table inheritance.

Figure 1.4d Illustrating some of the object features of SQL. Creating tables based on some of the UDTs, and illustrating **table inheritance**.

```
(d) CREATE TABLE PERSON OF PERSON_TYPE
      REF IS PERSON_ID SYSTEM GENERATED;
CREATE TABLE EMPLOYEE OF EMPLOYEE_TYPE
      UNDER PERSON;
CREATE TABLE MANAGER OF MANAGER_TYPE
      UNDER EMPLOYEE;
CREATE TABLE STUDENT OF STUDENT_TYPE
      UNDER PERSON;
```

Contd..,

- **1.2.6. Specifying Relationships via Reference:**
 - Component attribute of one tuple may be a **reference** to a tuple of another table
 - Specified using keyword **REF**
 - Keyword **SCOPE**
 - Specify name of table whose tuples referenced
 - Similar to foreign key except it uses OID instead of PK.
 - **Syntax:** *ref_attribute REF (Type_Name) SCOPE (Referenced_Table)*

Contd..,

- Figure 1.4e Illustrating some of the object features of SQL. Specifying relationships using **REF** and **SCOPE**.

```
(e) CREATE TYPE COMPANY_TYPE AS (  
      COMP_NAME    VARCHAR (20),  
      LOCATION     VARCHAR (20));  
CREATE TYPE EMPLOYMENT_TYPE AS (  
      Employee REF (EMPLOYEE_TYPE) SCOPE (EMPLOYEE),  
      Company REF (COMPANY_TYPE) SCOPE (COMPANY) );  
CREATE TABLE COMPANY OF COMPANY_TYPE (  
      REF IS COMP_ID SYSTEM GENERATED,  
      PRIMARY KEY (COMP_NAME) );  
CREATE TABLE EMPLOYMENT OF EMPLOYMENT_TYPE;
```

Contd..,

- **1.2.6. Specifying Relationships via Reference: Path expressions**
 - SQL uses a **dot notation** to build **path expressions** that refer to the component attributes of tuples and row types.
 - if **r** is tuple (object) and **a** is a component attribute in that tuple, then **r.a** is the value of attribute a in that tuple.
 - However, for an attribute whose type is REF, the **dereferencing symbol** $\rightarrow$ is used.
 - if **r** is a reference to a tuple (object) and **a** is a component attribute in that tuple, then **r $\rightarrow$ a** is the value of attribute a in that tuple.

1.3. ODMG Object Model and Object Definition Language ODL

- A consortium of ODB vendors and users, called **ODMG (Object Data Management Group)**, proposed a standard that is known as the ODMG-93 or ODMG 1.0 standard.
- This was revised into ODMG 2.0, and later to ODMG 3.0.
- The standard is made up of several parts, including:
 - The object model,
 - The object definition language (ODL),
 - The object query language (OQL), and
 - The bindings to object-oriented programming languages.

Contd..,

- **1.3.1. Overview of the Object Model of ODMG:**
 - The ODMG object model is a standard data model for object databases.
 - Based on this data model **object definition language (ODL)** and **object query language (OQL)** are developed.
 - **Objects and literals** are the basic building blocks of the ODMG object model.

Contd..,

- **1.3.1. Overview of the Object Model of ODMG (continue.): Literal** -

In the ODMG object model,

- A literal is a value that does not have an object identifier.
- However, the value may have a simple or complex structure.
- There are three types of literals:
 1. **Atomic**: predefined; basic data type values. e.g., **short, float, boolean, char**)
 2. **Structured**: values that are constructed by type constructors. e.g., **date, struct, tuples, etc.**
 3. **Collection**: a collection of values or objects. E.g., **set, list, array, etc.**

Contd..,

- **1.3.1. Overview of the Object Model of ODMG (continue..): Object** - In the ODMG object model, object has following five aspects:
 1. **Identifier**: unique system-wide identifier
 2. **Name**: unique within a particular database; it is optional
 3. **Lifetime**: persistent vs. transient
 4. **Structure**: specifies how object is constructed by the type constructor and whether it is an atomic object
 5. **Creation**: Manner in an object can be created. E.g. new.
- Each object has state(property - attributes and relationships) and behavior (operation)
- In the ODMG object model, two way to specify the **object types** - **Class** and **interface**

Contd..,

- **1.3.1. Overview of the Object Model of ODMG (continue.): Interface -**

In the ODMG object model,

- An interface is a specification of the abstract behavior of an object type
- Specified using keyword **Interface** in Object Definition Language (ODL).
- **E.g.,**

```
interface Object
{
    ...
    object copy( );
    void delete( );
};
```

- **Non-instantiable:** Objects cannot be instantiated from an interface

Contd..,

- **1.3.1. Overview of the Object Model of ODMG (continue.): Class** - In the ODMG object model,
 - A class is a specification of both **state** and **behavior** of an object type.
 - That is, specification of a class includes its:
 - **Attribute** : Property; describes data in an object
 - **Relationship**: Specifies inter-object references
 - Relationship in inverse directions is specified using inverse keyword.
 - **Operation**: Operation name, argument types, return value
 - In addition, extent and key may also be included in a class.
 - Class is **Instantiable**, Hence, database objects are created based on the specified class.
 - Specified using keyword **class** in Object Definition Language (ODL).

Contd..,

- **Extents:**
 - An ODMG object can have an **extent** defined via a class declaration
 - Each extent is given a name and will contain all persistent objects of that class
 - For `Employee` class, for example, the extent is called `all_employees`
 - This is similar to creating an object of type `Set<Employee>` and making it persistent

Contd..,

- **Keys:**
 - One or more properties whose values are unique for each object in extent of a class
 - For the Employee class, the key is ssn
 - Thus each employee is expected to have a unique ssn
 - Keys can be composite, e.g.,
 - (key dnumber, dname)

Contd..,

- Figure 1.7 The attributes, relationships, and operations in a class definition.

```
class EMPLOYEE
(   extent      ALL_EMPLOYEES
    key         Ssn )
{
    attribute    string      Name;
    attribute    string      Ssn;
    attribute    date Birth_date;
    attribute    enum Gender{M, F} Sex;
    attribute    short      Age;
    relationship DEPARTMENT  Works_for
        inverse DEPARTMENT::Has_emps;
    void         reassign_emp(in string New_dname)
        raises(dname_not_valid);
};

class DEPARTMENT
(   extent      ALL_DEPARTMENTS
    key         Dname, Dnumber )
{
    attribute    string      Dname;
    attribute    short      Dnumber;
    attribute    struct Dept_mgr {EMPLOYEE Manager, date Start_date}
        Mgr;
    attribute    set<string>  Locations;
    attribute    struct Projs {string Proj_name, time Weekly_hours}
        Projs;
    relationship set<EMPLOYEE> Has_emps inverse EMPLOYEE::Works_for;
    void         add_emp(in string New_ename) raises(ename_not_valid);
    void         change_manager(in string New_mgr_name; in date
        Start_date);
};
```


Contd..,

- **Factory Objects:**
 - An object factory interface is used to generate individual objects via its operations
 - An example:

```
interface ObjectFactory
{
    Object new ( );
};
```
 - new() returns new objects with an object_id
 - One can create their own factory interface by inheriting the above interface
- **Operations** are applied to objects using the dot notation. For example, O.same_as(P) or O->same_as(P).

Contd..,

- **1.3.2. Inheritance in the Object Model of ODMG:** In the ODMG object model, two types of inheritance relationships exist:
 - **Behavior inheritance**
 - Also known as IS-A or interface inheritance
 - Specified by the colon (:) notation
 - Allow multiple inheritance.
 - **EXTENDS inheritance**
 - Specified by keyword **extends**
 - Inherit both state and behavior strictly among classes
 - Multiple inheritance via extends not permitted.

Contd..,

- **1.3.2. Inheritance in the Object Model of ODMG: Inheritance via : i.e., behavior inheritance – An Example:**

```
interface Shape
```

```
{
```

```
    ...
```

```
    float perimeter ( );
```

```
    float area( );
```

```
    ...
```

```
};
```

```
class Triangle: Shape
```

```
{
```

```
    attribute short side_1;
```

```
    attribute short side_2;
```

```
    ...
```

```
};
```

Contd..,

- **1.3.2. Inheritance in the Object Model of ODMG: Inheritance via extends**
– **An Example:**

Class Person

{

.....

attribute string name;

int age (in date DOB)

.....

};

class Employee extends Person

{

attribute string rank;

attribute float salary;

attribute string phone;

...

void promote (in string new_rank);

};

Contd.,

- **1.3.3. Built-in Interfaces and Classes in the Object Model:** In the object model, all objects inherit the basic interface operations (create, delete, etc.) of Object, as shown in Figure below.

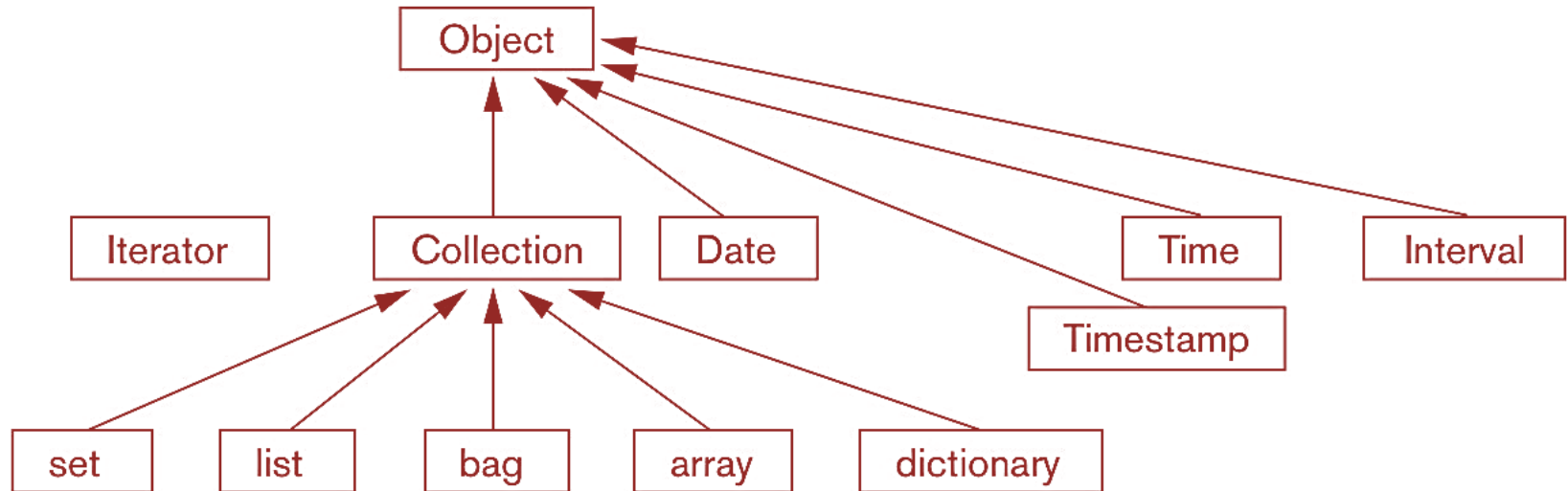


Figure 1.6 Inheritance hierarchy for the built-in interfaces of the object model.

Contd..,

- **1.3.3. Built-in Interfaces and Classes in the Object Model (continue..):**
 - A **collection** object includes the basic **collection** interface operations, for example:
 - `cardinality()`
 - `is_empty()`
 - `insert_element()`
 - `remove_element()`
 - `contains_element()`
 - `create_iterator()`

Contd..,

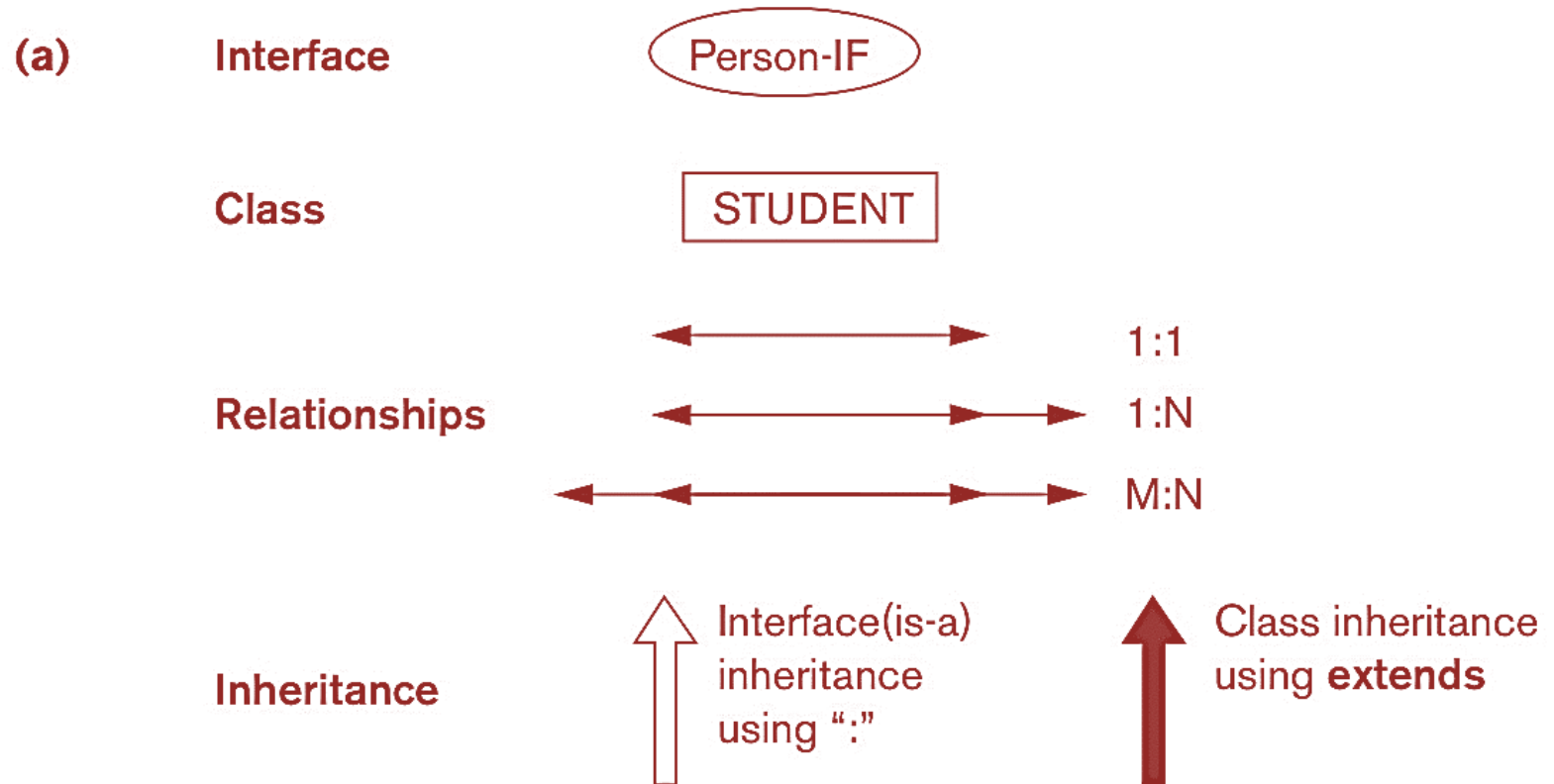
- **1.3.3. Built-in Interfaces and Classes in the Object Model (continue..):**
 - Collection objects are further specialized into types like a set, list, bag, array, and dictionary
 - Each collection type may provide additional interface operations, for example, a set provides:
 - `create_union( )`
 - `create_difference( )`
 - `is_subset_of( )`
 - `is_superset_of( )`
 - `is_proper_subset_of( )`

Contd..,

- **1.3.4. Object Definition Language ODL:**
 - Language used to create object specifications in the ODBs — that is, classes and interfaces.
 - More specifically, language used to create an object database schema.
 - Support semantic constructs of ODMG object model
 - Independent of any particular programming language.

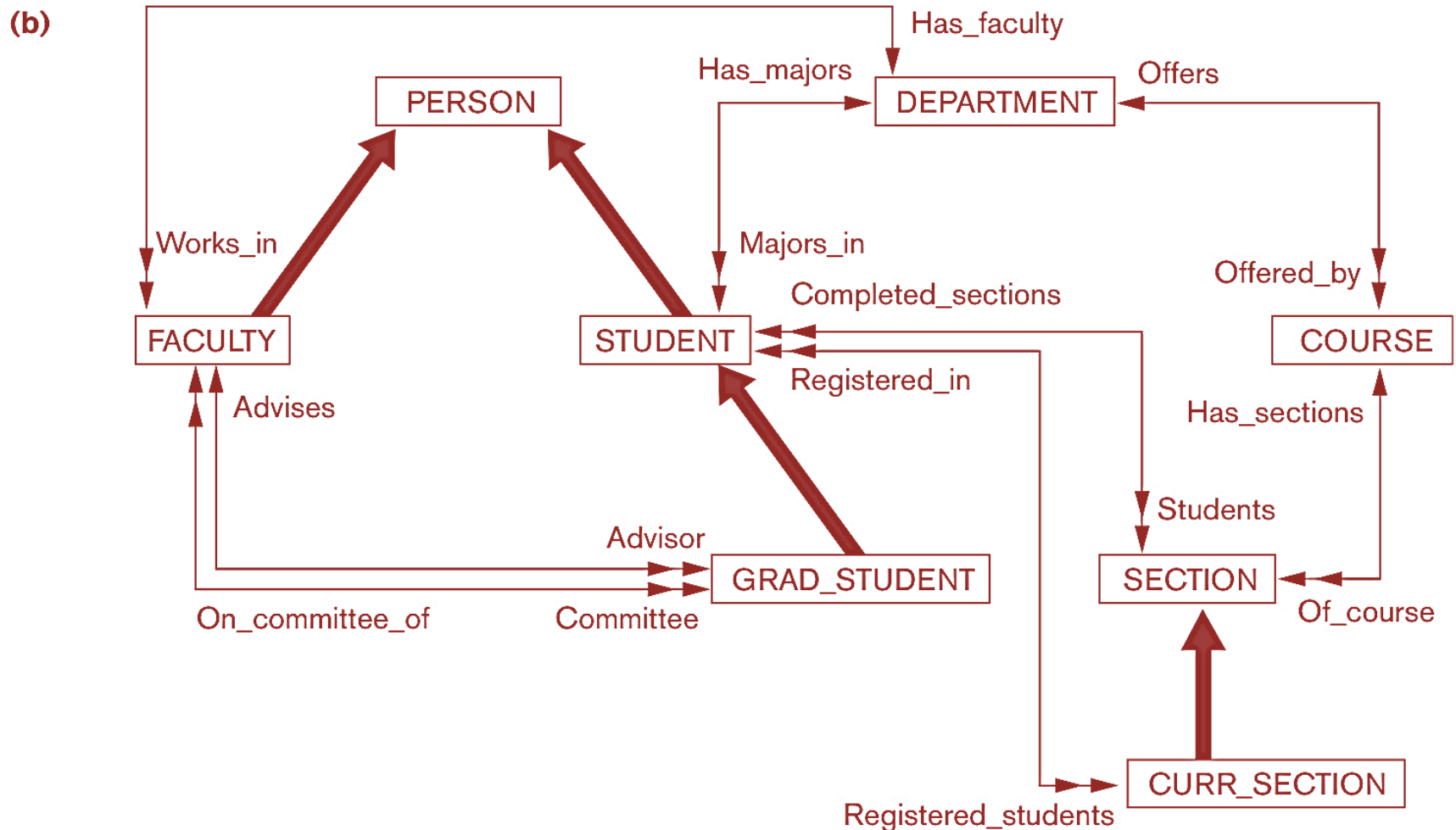
Contd.,

- Figure 1.9a An example of a database schema (UNIVERSITY database). **Graphical notation for representing ODL schemas.**



Contd.,

- Figure 1.9b An example of a database schema. A **graphical object database schema** for part of the UNIVERSITY database (GRADE and DEGREE classes are not shown).



Contd.,

- Figure 1.10 Possible ODL schema for the UNIVERSITY database in Figure 1.9(b).

```

class PERSON
(
  extent      PERSONS
  key         Ssn )
{
  attribute   struct Pname {   string  Fname,
                                string  Mname,
                                string  Lname }      Name;

  attribute   string          Ssn;
  attribute   date            Birth_date;
  attribute   enum Gender{M, F} Sex;
  attribute   struct Address { short   No,
                                string  Street,
                                short   Apt_no,
                                string  City,
                                string  State,
                                short   Zip }      Address;

  short      Age(); };
class FACULTY extends PERSON
(
  extent      FACULTY )
{
  attribute   string          Rank;
  attribute   float           Salary;
  attribute   string          Office;
  attribute   string          Phone;
  relationship DEPARTMENT    Works_in inverse DEPARTMENT::Has faculty;
  relationship set<GRAD_STUDENT> Advises inverse GRAD_STUDENT::Advisor;
  relationship set<GRAD_STUDENT> On_committee_of inverse GRAD_STUDENT::Committee;
  void        give_raise(in float raise);
  void        promote(in string new rank); };
  
```

Cont'd – next slide ->

Contd.,

- Figure 1.10 (continued) Possible ODL schema for the UNIVERSITY database in Figure 1.9(b).

```
class GRADE
(   extent GRADES )
{
    attribute      enum GradeValues{A,B,C,D,F,I, P} Grade;
    relationship    SECTION Section inverse SECTION::Students;
    relationship    STUDENT Student inverse STUDENT::Completed_sections; };

class STUDENT extends PERSON
(   extent      STUDENTS )
{   attribute    string      Class;
    attribute    Department   Minors_in;
    relationship  Department Majors_in inverse DEPARTMENT::Has_majors;
    relationship  set<GRADE> Completed_sections inverse GRADE::Student;
    relationship  set<CURR_SECTION> Registered_in INVERSE CURR_SECTION::Registered_students;
    void          change_major(in string dname) raises(dname_not_valid);
    float         gpa();
    void          register(in short secno) raises(section_not_valid);
    void          assign_grade(in short secno; IN GradeValue grade)
                    raises(section_not_valid,grade_not_valid); };
```

Cont'd – next slide ->

Contd.,

- Figure 1.10 (continued) Possible ODL schema for the UNIVERSITY database in Figure 1.9(b).

```
class DEGREE
{   attribute      string          College;
    attribute      string          Degree;
    attribute      string          Year; };

class GRAD_STUDENT extends STUDENT
(   extent          GRAD_STUDENTS )
{   attribute      set<Degree>     Degrees;
    relationship    Faculty advisor inverse FACULTY::Advises;
    relationship    set<FACULTY>   Committee inverse FACULTY::On_committee_of;
    void            assign_advisor(in string Lname; in string Fname)
                                raises(faculty_not_valid);
    void            assign_committee_member(in string Lname; in string Fname)
                                raises(faculty_not_valid); };

class DEPARTMENT
(   extent          DEPARTMENTS
    key             Dname )
{   attribute      string          Dname;
    attribute      string          Dphone;
    attribute      string          Doffice;
    attribute      string          College;
    attribute      FACULTY         Chair;
    relationship    set<FACULTY> Has_faculty inverse FACULTY::Works_in;
    relationship    set<STUDENT> Has_majors inverse STUDENT::Majors_in;
    relationship    set<COURSE> Offers inverse COURSE::Offered_by; };
```

Contd.,

- Figure 1.10 (continued) Possible ODL schema for the UNIVERSITY database in Figure 1.9(b).

```
class COURSE
(   extent      COURSES
    key         Cno )
{   attribute   string          Cname;
    attribute   string          Cno;
    attribute   string          Description;
    relationship set<SECTION> Has_sections inverse SECTION::Of_course;
    relationship <DEPARTMENT> Offered_by inverse DEPARTMENT::Offers; };

class SECTION
(   extent      SECTIONS )
{   attribute   short          Sec_no;
    attribute   string         Year;
    attribute   enum Quarter{Fall, Winter, Spring, Summer}
                Qtr;
    relationship set<Grade> Students inverse Grade::Section;
    relationship COURSE Of_course inverse COURSE::Has_sections; };

class CURR_SECTION extends SECTION
(   extent      CURRENT_SECTIONS )
{   relationship set<STUDENT> Registered_students
                inverse STUDENT::Registered_in
    void        register_student(in string Ssn)
                raises(student_not_valid, section_full); };
```

Contd..,

- **Interface Inheritance in ODL:** Following example illustrates interface inheritance in ODL

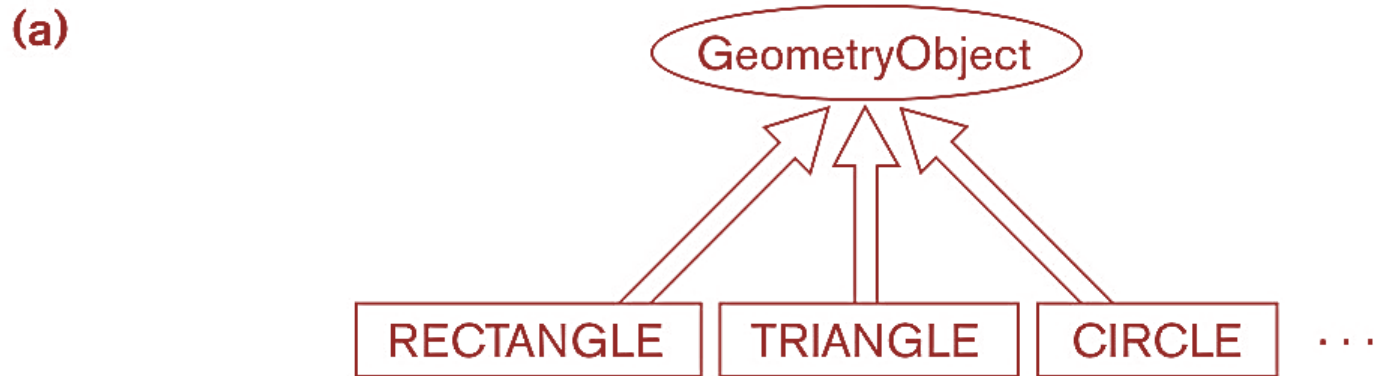


Figure 1.11a An illustration of interface inheritance via “:”. Graphical schema representation.

Contd..,

- Figure 1.11b An illustration of interface inheritance via “:”. Corresponding interface and class definitions in ODL.

```
(b) interface GeometryObject
{   attribute      enum      Shape{RECTANGLE, TRIANGLE, CIRCLE, ... }
    attribute      struct    Point {short x, short y} Reference_point;
    float          perimeter();
    float          area();
    void           translate(in short x_translation; in short y_translation);
    void           rotate(in float angle_of_rotation); };

class RECTANGLE : GeometryObject
(   extent      RECTANGLES )
{   attribute    struct    Point {short x, short y} Reference_point;
    attribute    short     Length;
    attribute    short     Height;
    attribute    float     Orientation_angle; };

class TRIANGLE : GeometryObject
(   extent      TRIANGLES )
{   attribute    struct    Point {short x, short y} Reference_point;
    attribute    short     Side_1;
    attribute    short     Side_2;
    attribute    float     Side1_side2_angle;
    attribute    float     Side1_orientation_angle; };

class CIRCLE : GeometryObject
(   extent      CIRCLES )
{   attribute    struct    Point {short x, short y} Reference_point;
    attribute    short     Radius; };

...
```


1.4. Object Database Conceptual Design

- **1.4.1. Differences between conceptual design of ODB and RDB:**

Major area of difference between ODB and RDB design

- **Relationships:** ODB uses OIDs (object references), RDB uses foreign keys (values)
- **Multivalued / Binary Relationships:** ODB allows collections and bidirectional references; RDB requires separate tables for M:N relationships
- **Inheritance:** ODB has built-in support, RDB has no built-in support.
- **Operations / Methods:** ODB defines methods early in class design; RDB adds triggers/procedures later.
- **Behavior:** ODB enforces encapsulation and predefined behaviors; RDB supports ad hoc queries and flexible operations
- **Design Approach:** ODB combines structure and operations from the start; RDB designs structure first, then operations

Contd..,

- **1.4.2. Mapping an EER Schema to an ODB Schema:**
 - Step1: Create ODL class for each EER entity type
 - Step2: Add relationship properties for each binary relationship
 - Step3: Include appropriate operations for each class
 - Step4: ODL class that corresponds to a subclass in the EER schema:
Inherits type and methods of its superclass in ODL schema
 - Step5: Weak entity types: Mapped same as regular entity types
 - Step6: Categories (union types) Difficult to map to ODL
 - Step7: An n -ary relationship with degree $n > 2$: Map into a separate class, with appropriate references to each participating class

Contd..,

- **Step 1:** Create an ODL class for each EER entity type or subclass
 - Multi-valued attributes are declared by sets, bags or lists constructors
 - Composite attributes are mapped into tuple constructors
 - Declare an extent for each class, and specify any key attributes as keys of the extent.
- **Step 2:** Add relationship properties or reference attributes for each binary relationship into the ODL classes participating in the relationship
 - Relationship cardinality: single-valued for 1:1 and N:1 directions; set-valued for 1:N and M:N directions
 - Relationship attributes: create via tuple **constructors**

Contd..,

- **Step 3:** Add appropriate operations for each class
 - Operations are not available from the EER schemas; original requirements must be reviewed
 - Corresponding **constructor** and **destructor** operations must also be added
- **Step 4:** Specify inheritance relationships via extends clause
 - An ODL class that corresponds to a sub-class in the EER schema inherits the types and methods of its super-class in the ODL schemas
 - Other attributes of a sub-class are added by following Steps 1-3

Contd..,

- **Step 5:** Map weak entity types in the same way as regular entities
 - Weak entities that do not participate in any relationships may alternatively be presented as **composite multi-valued attribute** of the owner entity type
- **Step 6:** Map categories (union types) to ODL
 - Declare a class to represent the category
 - Define 1:1 relationships between the category and each of its super-classes
- **Step 7:** Map **n-ary relationships** whose degree is greater than 2
 - Each relationship is mapped into a separate class with appropriate reference to each participating class

1.5. The Object Query Language OQL

- The **object query language OQL** is the query language proposed for the ODMG object model.
- It is designed to work closely with the programming languages for which an ODMG binding is defined, such as C++, Smalltalk, and Java.
- Hence, an OQL query embedded into one of these programming languages can return objects that match the type system of that language.
- Additionally, the implementations of class operations in an ODMG schema can have their code written in these programming languages.
- OQL's syntax is similar to SQL with additional features for objects

Contd..,

- **Simple OQL Queries:**

- Basic syntax: select...from...where...
- For Example:

```
SELECT          d.name  
  
FROM           d in departments  
  
WHERE          d.college = 'Engineering';
```

- An **entry point** to the database is needed for each query
- An extent name (e.g., departments in the above example) may serve as an entry point
- That is, named persistent object is used as Entry point to the database.

Contd..,

- **Iterator Variables:**

- Iterator variables are defined whenever a collection is referenced in an OQL query
- Iterator **d** in the previous example serves as an iterator and ranges over each object in the collection
- Syntactical options for specifying an iterator:
 - *d in departments*
 - *departments d*
 - *departments as d*

Contd..,

- **Data Type of Query Results:**

- The data type of a query result can be any type defined in the ODMG model
- A query does not have to follow the select...from...where... format
- A persistent name on its own can serve as a query whose result is a reference to the persistent object.
- For example,
 - *departments; whose type is set<Departments>*

Contd..,

- **Path Expressions:**

- A **path expression** is used to specify a path to attributes and objects in an entry point
- A path expression starts at a persistent object name (or its iterator variable)
- The name will be followed by zero or more dot connected relationship or attribute names
 - E.g., *departments.chair;*

Contd..,

- **Views as Named Objects:**
 - The **define** keyword in OQL is used to specify an identifier for a **named query**
 - The name should be unique; if not, the results will replace an existing named query
 - Once a query definition is created, it will persist until deleted or redefined
 - A view definition can include parameters

Contd..,

- **An Example of OQL View:**

- A view to include students in a department who have a minor:

define** has_minor(dept_name) **as

***select** s*

***from** s **in** students*

***where** s.minor_in.dname=dept_name*

- has_minor can now be used in queries

Contd..,

- **Single Elements from Collections:**

- An OQL query returns a collection
- OQL's element operator can be used to return a single element from a singleton collection that contains one element:

***element** (select d from d in departments*

***where** d.dname = 'Software Engineering');*

- If d is empty or has more than one elements, an **exception** is raised

Contd..,

- **Collection Operators:**

- OQL supports a number of aggregate operators that can be applied to query results
- The aggregate operators and operate over a collection and include
 - *min, max, count, sum, avg*
- count returns an integer; others return the same type as the collection type

- **An Example of an OQL Aggregate Operator:** To compute the average GPA of all seniors majoring in Business:

```
avg (select s.gpa from s in students  
where s.class = 'senior' and  
s.majors_in.dname = 'Business');
```

Contd..,

- **Membership and Quantification:**

- OQL provides membership and quantification operators:
 - (e **in** c) is true if e is in the collection c
 - (**for all** e **in** c: b) is true if all e elements of collection c satisfy b
 - (**exists** e **in** c: b) is true if at least one e in collection c satisfies b
- **An Example of Membership:** To retrieve the names of all students who completed CS101:

```
select s.name.fname  s.name.lname
from                s in students
where                'CS101' in
    (
        select c.name
        from c
        in s.completed_sections.section.of_course
    );
```

Contd..,

- **Ordered Collections:**

- Collections that are lists or arrays allow retrieving their **first**, **last**, and **ith** elements
- OQL provides additional operators for extracting a sub-collection and concatenating two lists
- OQL also provides operators for ordering the results
- **An Example of Ordered Operation:** To retrieve the last name of the faculty member who earns the highest salary:

```
first (select struct  
      (faculty: f.name.lastname,  
        salary f.salary)  
from f in faculty  
ordered by f.salary desc);
```


Contd..,

- **Grouping Operator:**

- OQL also supports a grouping operator called **group by**
- Similar to the corresponding clause in SQL
- Provides explicit reference to the collection of objects within each group or partition
- To retrieve average GPA of majors in each department having >100 majors:

select deptname, avg_gpa:

avg (select p.s.gpa from p in partition)

from s in students

group by deptname: s.majors_in.dname

having count (partition) > 100

- **Having clause:** *Used to filter partitioned sets*

Contd..,

- **Overview of the C++ Language Binding in the ODMG Standard (Extra):**
 - Specifies how ODL constructs are mapped to C++ constructs
 - Uses prefix d_ for class declarations that deal with database concepts
 - Template classes
 - Specified in library binding
 - Overloads operation new so that it can be used to create either persistent or transient objects

Thank You !

